

Knowledge Transfer Session

GO Backend & Frontend New Features

Presenter: Mustafa Africawala, Javier Alonso, Abdullah Alotaishan, Gopala Bhamidipati, Varun Narendra

Role: UCL IXN internships 2025

Duration: Three months



Project Overview & Deliverables



Backend

 [github.com](#)

GO API 2 Repository



We created a private repository as a copy of the the go-api. We developed four API endpoints corresponding to each feature (F) identified

- F1 Previous Crises Insights
- F2 Rapid Response Capacity Excel Generation
- F3 DREF Summary
- F4 DREF Situational Overview



Frontend

 [github.com](#)

GO Web App 2 Repository



We created a private repository as a copy of the go-web-app. We added changes to the Emergency Details page to further show the new data

We enhanced the Emergency Details page to display new AI-powered insights and data, adding:

- LessonCard for operational insights
- Key Figures display
- Contacts section
- Overview section
- Previous Crises summaries
- Rapid Response download section
- Situational Overview display
- New Emergency Operational Strategy tab



Testing

 [github.com](#)

Testcases Repository



We created a public repository that holds all of the testcases that were created to test all workflows. It holds all of the excel files created for Rapid Response as well as the shared excel file. Also, it holds all of the csv files created for the autoamted LLM tests.

F1 - Previous Crises Insights

What

New API endpoint that returns AI enhanced operational learnings based on geographical location and disaster type.

Why

Aid IFRC/NS users in the creation of emergency pages from scratch by enabling access to lessons learned from similar disasters in comparable geographical or operational contexts.



F1 - Previous Crises Insights

Key Components

1

Lesson Cards showing Insights

- We show all previous crises insights using custom lesson cards and the `useIfrcEvents` hook.

2

Retrieve Operational Learnings and Events

- We use an HTTP Client to access production data, and filter by country id and disaster type.

3

Create up to 6 Insights using Azure OpenAI

- We input OL, Events and Rapid Response Capacity Questions to the prompt.

Implementation Details



Key Files & Folders

`./app/src/hooks/domain/useIfrcEvents.ts` – fetches and formats previous crisis data\

`./per/ucl_research/ucl_views.py` - contains the view that was added to the `urls.py`

`./per/ucl_research/serializers.py` - contains response models.

`./per/ucl_research/ifrc_client.py` - fetches data from production IFRC API.

`./per/ucl_research/ops_learning_summary4.py` - contains all of the logic related to the prompt engineering.

`./per/ucl_research/evaluation/previous_crises_eval_script.py` - contains the LLM automated tests.

`./per/ucl_research/evaluation/results/*` - contains the csv result after running the tests.

API Endpoint

Method	GET
URL	/api/v1/ucl/previous-crises-insights
Params	country={id}&disaster_type={id}

F1- Previous Crises Insights

Prompt Engineering Key Logic

- **Mandatory Learning Merge Strategy**
 - "You MUST return a JSON array of high-quality insights, each merging between two and three distinct learnings into a single, cohesive narrative"
 - "Aim to MERGE learnings wherever possible – do not create single-learning insights unless no valid merge exists"
- **Disaster Type Matching Priority**
 - "Prioritise insights that have a matching disaster type between their learnings"
- **Operational Context Anchoring**
 - "Each insight must reference specific facts, examples, or operational details from the source learnings AND clearly name the country and disaster type"
 - "The tone should be practical and aimed at helping a current similar crisis"
- **Strict Uniqueness Enforcement**
 - "Use each Learning ID only once across ALL insights"
 - "Never repeat a Learning ID in more than one insight"
 - "If you cannot create more insights without reusing IDs, stop and output only the valid ones"
- **Zero Speculation Policy**
 - "Each insight MUST be based only on the provided learning content – NO invention or unsupported speculation"
 - "All claims must be directly traceable to the source learnings"
 - "Do NOT invent or speculate"
- **Source Attribution System**
 - "You MUST include the source of each learning you are referencing in metadata.operational_learning_source"
 - "Include the sources used in metadata.operational_learning_source with their id, code, and name"
- **Quality Over Quantity Principle**
 - "Quality is the top priority – fewer insights are fine if it avoids repetition or speculation"
 - "Produce no more than 6 insights in total, aiming for 4 to 5 as the ideal number for quality and coverage"
- **Narrative Integration Requirement**
 - "Each insight should weave the learnings together into one unified narrative – not just list them – showing how they connect and complement each other"
 - "Each insight must be at least 4 sentences long, rich with detail"
- **Template-Driven RR Question Generation**
 - "Generate RR questions for each insight. Match each insight to the most relevant 'Area' in the template JSON, generate 1-2 focused RR questions based on that Area's 'Critical Questions'"

F1- Previous Crises Insights

Lessons Learned

- Asking the LLM for *recommendations* increased the risk of hallucinations → reframed the task to generate Insights grounded in existing learnings.
- Allowing one learning to support multiple Insights led to repetition and the omission of fresh data, reducing overall coverage.
- LLMs can test for uniqueness; running a post-generation validation step helps filter duplicates and keep insights diverse.
- Forcing exactly 6 insights caused the model to stretch weak content → better to allow a flexible range (2 - 6) depending on the data.
- LLMs respond better when given stimuli → future prompts should include a human-written example of an insight document to improve alignment.

F2 - Rapid Response Capacity Questions

What

New API endpoint that creates a Rapid Response Capacity Questions (RRCQ) excel file with all of the questions answered using Operational Learning and LLMs

Why

Enable IFRC/NS users to react faster to newly created emergencies by automating the generation of the Rapid Response Capacity Questions.



F2 - Rapid Response Capacity Excel Generation

Key Components

- 1

Excel Template Retrieval

 - In the emergency page we use the `useRapidResponse` hook that calls backend API to fetch pre-filled rapid response Excel capacity assessment templates.
- 2

Retrieve Operational Learnings and Events

 - We use an HTTP Client to access production data, and filter by country id and disaster type.
- 3

RR Question Processing and Gap Detection

 - We load the RRCQ from our template.
- 4

Source Filtering and Ranking

 - We extract keywords from question text, area, and guiding questions
 - We score and rank all sources (events + ops-learning) by relevance
 - We apply diversification logic to select top 5 sources from different events
- 5

AI Response Generation

 - We build system prompt with strict source-only constraints and generate 3-4 bullet points with specific facts and references
- 6

Excel Export & BLOB Upload

 - We create a excel file with the desired structure, populate the questions answers with the AI response and Upload to blob storage and cache the result URL

Implementation Details

i

Key Files & Folders

`./app/src/hooks/domain/useRapidResponse.ts` – backend integration hook for Excel file fetching.

`./per/ucl_research/ucl_views.py` - contains the view that was added to the urls.py

`./per/ucl_research/serializers.py` - contains response models.

`./per/ucl_research/ifrc_client.py` - fetches data from production IFRC API.

`./per/ucl_research/ops_learning_summary4.py` - contains all of the logic related to the prompt engineering.

`./per/ucl_research/blob_upload.py` - uploads a file to Azure Blob Storage Service.

`./per/ucl_research/evaluation/rr_capacity_eval_script.py` - contains the LLM automated tests.

`./per/ucl_research/evaluation/results/*` - contains the csv result after running the tests.

API Endpoint

Method	GET
URL	/api/v1/ucl/rapid-response-capacity-excel
Params	id={emergency_id}

F2 - Rapid Response Capacity Questions Excel Generation

Prompt Engineering Key Logic

- **Strict Source-Only Information Policy**
 - "You are ONLY allowed to use information that is EXPLICITLY provided in the sources above"
 - "DO NOT attempt to infer, stretch, reframe, or generalise learnings beyond what is clearly supported by the sources"
 - "You are FORBIDDEN from using any information from your training data, general knowledge, or any other source"
- **Specific Output Format Requirements**
 - "Format: Label: analysis with specific facts (Reference: CODE - Event, Date)"
 - "Generate 3-4 bullets with diverse analytical perspectives"
 - "Plain text only, no markdown"
 - "Each bullet should include specific facts (numbers, dates, places, named units) from sources"
- **Analytical Perspective Diversification**
 - "Vary conclusions across bullets (strengths, contradictions, operational deltas)"
 - "Diversify analytical lenses: legal review, FR metrics, ops-learning, contacts' statements"
- **Fallback Response Enforcement**
 - "If the sources do not contain enough information to answer the question, you MUST respond with: 'Enough source is not available to answer this question'"
 - "If insufficient source data, respond: 'Enough source is not available to answer this question'"
- **Source Attribution and Validation**
 - "Tie each bullet to specific field report/appeal IDs with exact figures from sources"
 - "Ensure appeal codes match country context from sources"
 - Appeal code validation: Reject codes not present in source data
- **Emergency Response Specialist Persona**
 - "You are an IFRC emergency response specialist conducting rapid response capacity assessment"
- **Absolute Information Restrictions**
 - "NEVER create, invent, infer, assume, or generate ANY information not directly stated in sources"
 - "NEVER use information from your training data or general knowledge"
 - "Cross-check all facts against provided sources only"
- **Multi-Source Synthesis Requirement**
 - "When multiple sources are provided, synthesize information from all of them. Do not rely exclusively on the most detailed source"
- **Response Validation Rules**
 - "Cross-check all facts against provided sources only"
 - "Ensure appeal codes match country context from sources"

F2 - Rapid Response Capacity Questions Excel Generation

Lessons Learned

- Do not send the template Excel file directly; instead, extract questions into a JSON object or structured data for easier LLM prompting.
- A pre-processing ranking step is required to choose accurate and appropriate sources.
- A post-generation validation step helps filter duplicates and ensure insights remain diverse.
- Testing this feature can be computationally expensive due to LLM credit usage.
- Giving the LLM an example output can be fruitful for the final outcome

F3 - DREF Summary

What

New API endpoint that summarises the Operational Strategy and Economic Sectors from the latest Operational Update of the related DREF of an Emergency. It also returns DREF type and onset

Why

Give IFRC/NS users more information from any DREF available in the emergency page.



F3 - DREF Summary

Key Components

- 1

New Operational Strategy Tab

 - Feature added in the emergency page. Contains an operational strategy summary and economic sectors accordeon lists.
 - `usePerDrefSummary` retrieves AI-generated DREF summaries from the backend.
- 2

Retrieve Field Reports, Events and DREF data

 - We use an HTTP Client to access production data and `dref_utils.py` to access the DREF dumps.
- 3

Create Operational Strategy Summary

 - We use key fields from the DREF to leverage LLMs to generate a summary that has key facts.
- 4

DREF Source Grouping

 - We identify and group each of the DREF economic sectors relevant data for further processing.
- 5

Create Economic Sectors Summaries

 - We use each of the sectors relevant data to generate Needs and Intervention summaries using LLMs.

Implementation Details

- i

Key Files & Folders

 - `./app/src/hooks/domain/usePerDrefSummary.ts` - data fetching hook
 - `./per/ucl_research/ucl_views.py` - contains the view that was added to the `urls.py`
 - `./per/ucl_research/serializers.py` - contains response models
 - `./per/ucl_research/ifrc_client.py` - fetches data from production IFRC API
 - `./per/ucl_research/ops_learning_summary4.py` - contains all of the logic related to the prompt engineering
 - `./per/dref_temp/*` - contains DREF JSON dumps and wrapper python files to access/filter the data.
 - `./per/ucl_research/evaluation/operational_eval_script.py` - contains the LLM automated tests.
 - `./per/ucl_research/evaluation/sector_summary_eval_script.py` - contains the LLM automated tests.
 - `./per/ucl_research/evaluation/results/*` - contains the csv result after running the tests.

API Endpoint

Method	GET
URL	/api/v1/ucl/dref-summary
Params	id={emergency_id}

F3 - DREF Summary

Prompt Engineering Key Logic

- **IFRC Expert Analyst Persona (Both)**
 - "You are an IFRC expert analyst specializing in DREF operations"
 - "Analyze DREF data and provide clear, actionable insights for humanitarian response planning"
 - "Use professional, analytical writing accessible to IFRC staff and National Society personnel"
- **Structured 3-Line Operational Summary**
 - "Line 1: Overall objective of the operation"
 - "Line 2: Strategic rationale and approach"
 - "Line 3: Key operational details (target population, timeline, budget)"
 - "Exactly 3 lines, one complete sentence each"
- **Specific Formatting Requirements (Operational Strategy)**
 - "Plain text format (not JSON)"
 - "Include specific numbers and details"
 - "Focus on operation_objective and response_strategy fields"
 - "When mentioning people affected, include households affected in parentheses if available"
- **Concise Needs Summary Requirements (Economic Sectors Focus)**
 - "Maximum 2 sentences"
 - "Include specific needs and vulnerabilities"
 - "Focus on humanitarian gaps and operational requirements"
 - "No extra spaces or line breaks"
- **Solution-Focused Intervention Summaries (Economic Sectors Focus)**
 - "Write a concise sentence focusing on the specific solution and measurable outcomes, not repeating the problem statement"
 - "Be direct and solution-focused"
 - "Focus on what will be done and the impact, not what the problems are"
 - "Include target numbers and specific actions"
- **Precision and Accuracy Standards (Both)**
 - "Be extra careful handling any facts or numbers with precision"
 - "Avoid repeating needs summary content"
 - "Single flowing sentence"
- **Example-Driven Prompting**
 - "Operational Strategy: 'The operation aims to provide emergency assistance to 5,000 flood-affected people (1,200)...'"
 - "Economic Sectors - Needs: 'The affected population faces critical water and sanitation...'"
 - "Economic Sectors - Intervention: 'The intervention will establish 12 water distribution...'"

F3 - DREF Summary

Lessons Learned

- Understand all DREF properties before integration; some are more important than others.
- Always select the latest version of the DREF before generating summaries.
- For operational strategy, prioritize the objective and rationale fields.
- For sectors, skip the *National Society Actions* list and focus on *Planned Interventions* and *Needs Identified*.
- Impersonating an IFRC user and providing examples in the prompt produced stronger outputs.

F4 - DREF Situational Overview

What

New API endpoint that summarises the event scope and event description of the DREF's latest Operational Update related to the emergency.

Why

Give IFRC/NS users a more in-depth situational overview of the emergency.



F4 - DREF Situational Overview

Key Components

- 1

Update to Situational Overview

 - This endpoint will work as a complement to the existing Situational Overview field of the emergency page.
 - `usePerDrefSituationalOverview` hook requests the PER-DREF situational overview for a given `dref_id`.
- 2

Retrieve Field Reports, Events and DREF data

 - We use an HTTP Client to access production data and `dref_utils.py` to access the DREF dumps.
- 3

Create Situational Overview Summary

 - We use key fields from the DREF to leverage LLMs to generate a summary that has key facts.

Implementation Details

- i

Key Files & Folders

 - `./app/src/hooks/domain/usePerDrefSituationalOverview.ts` – data fetching hook
 - `./per/ucl_research/ucl_views.py` - contains the view that was added to the `urls.py`
 - `./per/ucl_research/serializers.py` - contains response models
 - `./per/ucl_research/ifrc_client.py` - fetches data from production IFRC API
 - `./per/ucl_research/ops_learning_summary4.py` - contains all of the logic related to the prompt engineering
 - `./per/dref_temp/*` - contains DREF JSON dumps and wrapper python files to access/filter the data.
 - `./per/ucl_research/evaluation/situational_overview_eval_script.py` - contains the LLM automated tests.
 - `./per/ucl_research/evaluation/results/*` - contains the csv result after running the tests.

API Endpoints

Method	GET
URL	/api/v1/ucl/dref-situational-overview
Params	id={emergency_id}

F4 - DREF Situational Overview

Prompt Engineering Key Logic

- **IFRC Expert Analyst Persona**
 - "You are an IFRC expert analyst specializing in DREF operations"
 - "Analyze DREF data and provide clear, actionable insights for humanitarian response planning"
 - "Use professional, analytical writing accessible to IFRC staff and National Society personnel"
- **Output Format Requirements**
 - "Return only the 5-line paragraph without additional formatting or explanations"
- **Decision-Maker Orientation**
 - "Ensure the paragraph provides a clear situational understanding for decision-makers"
- **Comprehensive Writing Guidelines**
 - "Each line should be a complete, well-structured sentence"
 - "Maintain consistency and flow between all 5 lines"
 - "Use professional humanitarian language"
 - "Focus on factual information from the provided data"
- **5-Line Situational Overview Structure**
 - "Create a comprehensive 5-line situational overview paragraph based on the provided DREF operational update data"
 - "Lines 1-3: Focus on summarizing the event_description and event_scope from the latest operational update"
 - "Lines 4-5: Focus on summarizing the operation_objective and response_strategy"

F4 - DREF Situational Overview

Lessons Learned

- For situational overview, prioritize the event scope and event description fields.
- Instructing the prompt to define the meaning of each line in a paragraph improves the overall outcome.
- Fields on changes between DREF operational updates are not reliable or coherent enough for effective LLM use.

Powering the Frontend

Under the Hood

Tech Stack:

- React + TypeScript
- React Router
- IFRC GO UI library (@ifrc-go/ui)
- CSS Modules
- Custom hooks (useRequest)
- 118n JSON per view

New Components

1 Overview

Concise disaster summary with key details like severity, type, dates, visibility, and assistance flags.

2 Key Figures

At-a-glance numeric KPIs with data sources.

3 Contacts

Grouped emergency contacts with email and phone links.

4 Previous Crises

AI-powered operational learnings with expand/collapse and source details.

5 Rapid Response

Pre-filled Excel capacity questions for quick download.

6 Situational Overview

Rich HTML summary with DREF metadata.

7 Operational Strategy

DREF summary, sectoral needs, indicators, and budget file teaser.

UI Enhancements

1 Navigation

Added Operational Strategy tab and route to the Emergency navigation.

2 Data Hooks

New hooks with caching (10-min TTL) for IFRC events, DREF summary, situational overview, and rapid response.

3 Accessibility

Keyboard support on cards/downloads, ARIA roles, and focus states.

4 Responsive Design

Fully responsive grids and flex layouts, consistent spacing and typography, hover states.

5 Internationalization

118n keys for all new sections.

Powering the Frontend

Component Structure

- 1** `app/src/App/routes/index.tsx`
Adds the emergencyOperationalStrategy route and tab.
- 2** `app/src/views/EmergencyDetails/index.tsx`
Orchestrates data and passes props to subsections.
- 3** **Subviews**
Overview, KeyFigures, Contacts, PreviousCrises, RapidResponse, SituationalOverview, and EmergencyOperationalStrategy/OperationalStrategy page for DREF content.

Key Components

- 1** `EmergencyDetails/PreviousCrises/index.tsx`
Renders the AI learnings grid.
- 2** `components/LessonCard/index.tsx`
Expandable insight card with source links.
- 3** `EmergencyDetails/RapidResponse/index.tsx`
Download card for the Excel capacity questions.
- 4** `EmergencyDetails/SituationalOverview/index.tsx`
HTML summary with DREF metadata pill.
- 5** `EmergencyOperationalStrategy/OperationalStrategy/index.tsx`
DREF summary, sectors, indicators, and budget preview.

State Management

- 1** **Local UI State**
showAll, expanded (Set) in PreviousCrises; key handlers for Enter/Space.
- 2** **Async State**
useRequest hook with { response, pending, error, refetch }.
- 3** **Caching**
In-memory Map with timestamps in hooks (useIfrcEvents, usePerDrefSummary, useRapidResponse).
- 4** **Downloads**
Programmatic click with file_url.

CSS Modules

One styles.module.css per view; grid/flex layouts; mobile breakpoints.

Uses GO UI CSS variables (colors, spacing, radius) for theme consistency.

Maintenance Notes

Dependencies

Package	Version
openai	latest
httpx (transitive dependency of openai)	latest from openai
azure-identity	latest
django-storages[s3,azure]	1.14.5

Configuration



Environment Variables

`AZURE_OPENAI_ENDPOINT` - URL for your Azure OpenAI service instance

`AZURE_OPENAI_KEY` - Authentication key for Azure OpenAI API access

`AZURE_OPENAI_DEPLOYMENT_NAME` - Name of your deployed model (e.g., gpt-4, gpt-35-turbo)

`AZURE_ACCOUNT_NAME` - Azure Storage account name

`AZURE_ACCOUNT_KEY` - Azure Storage account access key

`AZURE_CONTAINER` - Azure Blob Storage container name

Config Files

`./env` - all environment variables are set up here.

`./main/urls.py` - all views are wired up here.

`./pyproject.toml` - all dependencies are listed here.

Next Steps & Recommendations



Suggested Improvement FRONTEND

Refactor `usePerDrefSummary` and `usePerDrefSituationalOverview` to remove hardcoded `id` values and accept dynamic parameters from route or component props.



Suggested Improvement FRONTEND

Implement skeleton loaders and shimmer effects for all new data-driven sections (Previous Crises, Rapid Response, DREF Summary, Situational Overview) to improve perceived performance.



Suggested Improvement BACKEND

Update all of the logic related to the DREF Situational Overview and DREF Summary views to use the databases instead of the `dref_temp` directory.

Suggested Improvement BACKEND

Don't use the IFRC Client, use the database layer directly to fetch necessary data from events, field reports, and operational data.

Suggested Improvement BACKEND

We can further filter previous related emergencies by location using the postgres capabilities, not just by country.

Suggested Improvement BACKEND

We were resource constrained and used the same GPT 4o mini engine for both the workflows and testing. It would be ideal to use something with more resources for testing.

Suggested Improvement BACKEND

All workflows are not using any kind of translations.



Contact Information:

Email Addresses

varun.narendra.24@ucl.ac.uk

javier.lopez.24@ucl.ac.uk

abdullah.alotaishan.24@ucl.ac.uk

gopala.bhamidipati.24@ucl.ac.uk

mustafa.africawala.24@ucl.ac.uk

GitHubs



 GitHub 
mustafa3252 – Overview
Agentic AI developer, Master's student @UCL. mustafa3252 has 38...



 GitHub 
Javiseeker – Overview
Electronics Engineer with experience in Technical Leadership. Mainly...



 GitHub 
varunnarendra – Overview
varunnarendra has 18 repositories available. Follow their code on GitHub.



 GitHub 
gopala01 – Overview
gopala01 has 13 repositories available. Follow their code on GitHub.



 GitHub 
Alotaishanab – Overview
Alotaishanab has 9 repositories available. Follow their code on GitHub.



[Issue 1] Hardcoded `id` in `usePerDrefSummary`

- Description:** The hook currently uses a fixed `id` value when calling the DREF Summary API, which prevents dynamic fetching based on the selected operation.
- Impact:** All calls return the same DREF Summary data regardless of user selection.
- Workaround:** Temporarily pass the desired `id` directly in the hook until code is refactored to accept it as a parameter.

[Issue 2] Hardcoded `id` in `usePerDrefSituationalOverview`

- Description:** The hook always uses the hardcoded value `6955` when calling the Situational Overview API.
- Impact:** UI displays the same situational overview for all emergencies.
- Workaround:** None for now, aside from manually changing the `id` in code. Refactor needed to replace hardcoded `id` with the `eventId` parameter.



Questions?

Feel free to ask about any aspect of the implementation